

COMPLETE\_PROJECT\_DOCUMENTATION.md



# Smart Transit Companion - Complete Project Documentation

## Full Project Idea, Architecture & Codebase Understanding

---



### Table of Contents

1. [Project Vision & Concept](#)
  2. [Problem Statement](#)
  3. [Solution Overview](#)
  4. [Technology Architecture](#)
  5. [AI Agent System Deep Dive](#)
  6. [Codebase Structure & Flow](#)
  7. [Database Architecture](#)
  8. [API Communication Flow](#)
  9. [Mobile App User Journey](#)
  10. [Admin Dashboard Features](#)
  11. [Key Implementation Details](#)
  12. [SLAIC 2025 Competition Alignment](#)
- 



## Project Vision & Concept

### Big Idea

Create an **AI-powered intelligent transportation companion** specifically designed for Sri Lankan commuters that transforms the chaotic, unpredictable nature of local public transportation into a seamless, optimized travel experience using cutting-edge multi-agent AI technology.

### What Makes This Project Unique?

## 1. 10 Specialized AI Agents Working Together

- Not just one AI, but a **coordinated team of 10 specialized agents**
- Each agent handles a specific aspect of route planning
- Orchestrated by **LangGraph** for intelligent workflow management
- Real-time collaboration and decision-making

## 2. Sri Lankan-First Design

- Deep integration with local transit systems (Buses, Trains, Tuk-tuks)
- Understands local context, culture, and travel patterns
- Multilingual support (English, Sinhala සිංහල, Tamil தமிழ்)
- Community-driven data validation

## 3. Production-Ready Full-Stack Platform

- Enterprise-grade backend (FastAPI + MongoDB + Redis)
- Professional admin dashboard (Next.js 15)
- Cross-platform mobile app (React Native + Expo)
- Real-time updates via WebSocket

---

# Problem Statement

## Transportation Challenges in Sri Lanka

### For Daily Commuters:

#### 1. Unpredictable Transit Times

- Buses don't follow strict schedules
- Train delays are common
- Traffic congestion is unpredictable

#### 2. Information Fragmentation

- No single source for all transport options
- Fare information is inconsistent
- Real-time updates are lacking

#### 3. Complex Multi-Modal Journeys

- Combining bus, train, and tuk-tuk is complicated
- Optimal transfer points are unclear
- Cost optimization requires local knowledge

#### 4. Language Barriers

- Most apps are English-only
- Rural commuters need local language support
- Cultural context is missing

### For Transportation Ecosystem:

#### 1. Lack of Data-Driven Insights

- No analytics on passenger patterns
- Disruption reporting is informal
- Service improvements are reactive

#### 2. Poor Community Integration

- Crowdsourced information is untapped
- Local knowledge isn't leveraged
- Safety concerns aren't addressed systematically



## Solution Overview

### How Smart Transit Companion Solves These Problems

#### 1. Multi-Agent AI Route Planning

User Request → 10 AI Agents → Optimized Route

Example Flow:

"I need to go from Colombo Fort to Kandy"

↓

Input Processing Agent → Validates request, loads preferences

↓

Mode Router Agent → Determines multi-modal routing needed

↓

Transit Aggregation Agent → Finds bus + train combinations

↓  
Fare Calculation Agent → Calculates exact costs  
↓  
Fare Optimization Agent → Finds cheapest alternatives  
↓  
Preference Analysis Agent → Personalizes based on history  
↓  
Local Knowledge Agent → Adds Sri Lankan context  
↓  
Disruption Monitor Agent → Checks for delays/traffic  
↓  
Route Optimization Agent → Ranks & selects best route  
↓  
Final Response → "Take Train #1005 (LKR 180), arrives 3:45 PM"

## 2. Real-Time Intelligence

- **Live Disruption Monitoring:** AI analyzes traffic, weather, accidents
- **Predictive Analytics:** Anticipates delays before they happen
- **Dynamic Rerouting:** Suggests alternatives in real-time
- **Community Alerts:** Crowdsourced updates from other users

## 3. Cost Optimization

- **Fare Comparison:** All transport modes side-by-side
- **Cheapest Route Algorithm:** AI finds lowest-cost combinations
- **Time vs Cost Trade-offs:** User can balance speed vs savings
- **Savings Tracker:** Shows money saved over time

## 4. Cultural & Linguistic Integration

- **Multilingual Interface:** Full support for English, Sinhala, Tamil
- **Local Landmarks:** Uses familiar place names
- **Cultural Context:** Understands festivals, peak hours, customs
- **Accessibility:** Support for differently-abled travelers



## Technology Architecture

## System Architecture Overview

### USER INTERFACES



Mobile App

React Native + Expo

- Route Planning UI
- AI Agent Visualization
- RAG Chatbot
- Community Reports



Admin Dashboard

Next.js 15 + React 19

- User Management
- Analytics Dashboard
- System Monitoring
- Push Notifications



### API GATEWAY (FastAPI)



Authentication (JWT)



CORS Middleware



Error Handling



Rate Limiting (Redis)



Request Logging



WebSocket Support



### BUSINESS LOGIC LAYER



MULTI-AGENT AI SYSTEM (LangGraph Orchestration)

1. Input Processing Agent
2. Mode Router Agent
3. Standard Route Agent
4. Transit Aggregation Agent
5. Fare Calculation Agent
6. Fare Optimization Agent ★ (SLAIC 2025)
7. User Preference Analysis Agent
8. Local Knowledge Agent
9. Disruption Monitoring Agent
10. Route Optimization Agent

## RAG CHATBOT SYSTEM (ChromaDB + Google Gemini)

- Vector Database (50+ transit documents)
- Semantic Search (ChromaDB)
- LLM Integration (Gemini 2.0 Flash)
- Multilingual Support

## SUPPORTING SERVICES

- Google Maps API Integration
- Weather Service (OpenWeather)
- Sri Lankan Transit Data Service
- Disruption Intelligence Service
- Community Report Service
- Push Notification Service



## DATA LAYER

### MongoDB Atlas

- 17 Collections
- User data
- Route history
- Community reports

### Redis Cache

- Session management
- Rate limiting
- Real-time data

### ChromaDB (Vectors)

- Transit knowledge base
- Embeddings
- Semantic search

### Google Cloud

- API credentials
- Maps & Gemini AI



## EXTERNAL INTEGRATIONS

### Google Maps API

### Sri Lanka Railways

### OpenWeather API

### National Transport Commission



# AI Agent System Deep Dive

## LangGraph Workflow Implementation

**File: backend/app/services/workflow.py**

The workflow orchestrates all 10 agents using a state machine:

```
def create_travel_agent_workflow():
    workflow = StateGraph(TravelState)

    # Add all 10 agent nodes
    workflow.add_node("input_processing", input_processing_node)
    workflow.add_node("mode_router", mode_router_node)
    workflow.add_node("standard_route", standard_route_node)
    workflow.add_node("transit_route_aggregation", transit_route_aggregation_node)
    workflow.add_node("fare_calculation", fare_calculation_node)
    workflow.add_node("fare_optimization", fare_optimization_node) # SLAIC 2025
    workflow.add_node("user_preference_analysis", user_preference_analysis_node)
    workflow.add_node("local_knowledge_agent", local_knowledge_agent_node)
    workflow.add_node("disruption_monitoring", disruption_monitoring_node)
    workflow.add_node("route_optimization", route_optimization_node)

    # Define execution flow with conditional routing
    workflow.add_edge(START, "input_processing")
    workflow.add_edge("input_processing", "mode_router")

    # Conditional: Simple vs Complex routes
    workflow.add_conditional_edges(
        "mode_router",
        should_use_multi_agent,
        {
            "standard_processing": "standard_route",
            "transit_processing": "transit_route_aggregation"
        }
    )

    # Sequential execution for optimization
    workflow.add_edge("standard_route", "fare_calculation")
    workflow.add_edge("fare_calculation", "fare_optimization")
    workflow.add_edge("fare_optimization", "user_preference_analysis")
    workflow.add_edge("user_preference_analysis", "local_knowledge_agent")
    workflow.add_edge("local_knowledge_agent", "route_optimization")
    workflow.add_edge("route_optimization", "disruption_monitoring")
    workflow.add_edge("disruption_monitoring", END)

    return workflow.compile()
```

## Agent Details

### 1. Input Processing Agent

**File:** backend/app/services/agent\_nodes.py:input\_processing\_node()

**Purpose:** Validates user input and loads preferences

**What it does:**

- Validates source and destination addresses
- Retrieves user travel preferences from MongoDB
- Initializes the workflow state
- Handles errors gracefully

**Example:**

```
Input: "Colombo Fort" → "Kandy"
↓
Validates: ✓ Both locations exist
Loads User Preferences:
  - Language: Sinhala
  - Preferred Transport: Train, Bus
  - Accessibility: None
  - Max Budget: LKR 500
↓
Output: Validated state with user context
```

### 2. Mode Router Agent

**Purpose:** Determines routing strategy (simple vs multi-modal)

**Decision Logic:**

```
if mode in ["driving", "two_wheeler", "tuk_tuk", "uber"]:  
    → Use standard_route_agent (Google Maps only)  
elif mode in ["transit", "bus", "train"]:  
    → Use transit_aggregation_agent (Multi-modal)  
elif distance > 50km:  
    → Multi-agent processing required  
else:  
    → Standard processing
```

### 3. Standard Route Agent

**File:** backend/app/services/agent\_nodes.py:standard\_route\_node()

**Purpose:** Handles simple routes using Google Maps

**Integration:**

- **Google Maps Directions API**
- Requests multiple alternatives
- Includes traffic data
- Real-time duration estimates

**Example:**

```
Request: Colombo → Negombo (driving)  
↓  
Google Maps API Call:  
- Route 1: Colombo-Negombo Rd (35 min, 37 km)  
- Route 2: Baseline Rd (42 min, 41 km)  
- Route 3: A3 Highway (38 min, 39 km)  
↓  
Returns: 3 alternative routes with steps
```

### 4. Transit Route Aggregation Agent

**File:** backend/app/services/  
agent\_nodes.py:transit\_route\_aggregation\_node()

**Purpose:** Combines multiple transport modes

**What it does:**

- **Queries Google Maps** for transit directions
- **Identifies transfer points** between modes
- **Combines bus, train, and walking** segments
- **Optimizes connection timing**

### Example:

```
Request: Colombo Fort → Kandy (transit)
↓
Agent finds combinations:
1. Fort Railway → Kandy (Train #1005)
2. Fort → Pettah Bus Stand (Walk 10min) →
   Kaduwela Bus (Route 245) →
   Kandy Bus Stand (3h 15min)
3. Pettah → Kiribathgoda (Bus 138) →
   Kiribathgoda → Kandy (Train #1007)
↓
Returns: 3 multi-modal route options
```

## 5. Fare Calculation Agent

**File:** backend/app/services/agent\_nodes.py: fare\_calculation\_node()

**Purpose:** Calculates exact fares for all route segments

### Data Sources:

- **MongoDB Fare Database** (pre-loaded Sri Lankan fares)
- **Google Maps fare estimates**
- **Community-verified pricing**
- **Dynamic pricing APIs** (Uber/PickMe)

### Calculation Logic:

```
For each route step:
    if mode == "train":
        fare = query_train_fare_database(from, to, class)
    elif mode == "bus":
        fare = query_bus_fare_database(route_number, from, to)
    elif mode == "tuk_tuk":
        fare = estimate_tuktuk_fare(distance, time_of_day)
    elif mode == "uber":
        fare = get_uber_api_estimate(from, to)

    total_fare += fare
```

```
return {
    "total": total_fare,
    "currency": "LKR",
    "breakdown": [step_fares]
}
```

## 6. Fare Optimization Agent ★ (SLAIC 2025 Feature)

**File:** backend/app/services/agent\_nodes.py:fare\_optimization\_node()

**Purpose:** AI-powered cost minimization

**Advanced Features:**

- **Compares all alternatives** and finds cheapest
- **Identifies cost-saving transfers**
- **Time vs Cost trade-off analysis**
- **Shared ride optimization** (split tuk-tuk fares)

**AI Decision Making:**

```
routes = [Route1, Route2, Route3]

for route in routes:
    # Calculate cost score
    cost_score = 1 - (route.fare / max_fare)

    # Calculate time penalty
    time_penalty = (route.duration - min_duration) / max_duration

    # Weighted optimization
    optimization_score = (0.6 * cost_score) - (0.4 * time_penalty)

    route.optimization_score = optimization_score

# Recommend cheapest with acceptable time
best_route = max(routes, key=lambda r: r.optimization_score)

return {
    "recommended_route": best_route,
    "savings": max_fare - best_route.fare,
    "alternative_if_urgent": fastest_route
}
```

## 7. User Preference Analysis Agent

**File:** backend/app/services/  
agent\_nodes.py:user\_preference\_analysis\_node()

**Purpose:** Personalizes recommendations

**Learning Algorithm:**

```
# Analyze user history
user_history = get_user_route_history(user_id)

preferences = {
    "preferred_modes": count_mode_usage(user_history),
    "avg_budget": calculate_avg_fare(user_history),
    "common_times": analyze_travel_times(user_history),
    "accessibility_needs": user.accessibility_requirements
}

# Apply to current routes
for route in routes:
    # Boost score if matches preferences
    if route.primary_mode in preferences["preferred_modes"]:
        route.preference_score += 0.2

    if route.fare ≤ preferences["avg_budget"]:
        route.preference_score += 0.15

    # Penalty for non-accessible routes
    if user.needs_wheelchair_access and not route.is_accessible:
        route.preference_score -= 0.5

return sorted_routes_by_preference
```

## 8. Local Knowledge Agent

**File:** backend/app/services/agent\_nodes.py:local\_knowledge\_agent\_node()

**Purpose:** Adds Sri Lankan context and insights

**Knowledge Sources:**

- Google Gemini LLM for context generation
- Local landmarks database
- Cultural event calendar
- Peak hour patterns
- Destination insights

**Example Output:**

Input: Route to Temple of the Tooth, Kandy

↓

Agent adds context:

```
{
  "destination_insights": "Temple of the Tooth is a sacred Buddhist site.
                           Dress modestly (cover shoulders and knees).
                           Remove shoes before entering.
                           Peak pilgrimage times: 6-8 AM, 6-8 PM.",
  "travel_tips": "Train to Kandy offers scenic mountain views.
                  Arrive early for good seats.
                  Station is 2km from temple - tuk-tuks available at LKR 200-300.",
  "local_landmarks": "Pass by Peradeniya Botanical Gardens (worth a stop!)",
  "cultural_notes": "Poya Day (full moon) = temple very crowded,
                    plan extra time"
}
```

## 9. Disruption Monitoring Agent

**File:** backend/app/services/agent\_nodes.py:disruption\_monitoring\_node()

**Purpose:** Real-time disruption analysis and alerts

**Intelligent Features:**

- AI-powered severity assessment
- Predictive delay estimation
- Alternative route suggestions
- Multi-source data fusion

**Data Sources:**

```
disruptions = []

# 1. Community Reports (MongoDB)
community_reports = get_active_community_reports(route.area)
disruptions.extend(community_reports)

# 2. Weather API
weather = get_weather_for_route(route.coordinates)
if weather.severity ≥ "moderate":
    disruptions.append({
        "type": "weather",
        "description": f"Heavy rain expected along route",
        "severity": weather.severity,
        "impact": "15-30 min delay"
```

```
    })

# 3. Google Maps Traffic (real-time)
traffic = google_maps_tool.get_traffic_data(route.polyline)
if traffic.delay_minutes > 10:
    disruptions.append({
        "type": "traffic",
        "description": "Heavy traffic on A1 highway",
        "delay": traffic.delay_minutes
    })

# 4. AI Analysis (Gemini)
ai_analysis = llm_summarizer.analyze_disruption_impact(
    disruptions, route, user_context
)

return {
    "disruptions": disruptions,
    "total_delay_estimate": sum(delays),
    "ai_analysis": ai_analysis,
    "recommended_action": "Consider alternative route via B route"
}
```

## 10. Route Optimization Agent

**File:** backend/app/services/agent\_nodes.py:route\_optimization\_node()

**Purpose:** Final ranking and selection of best route

**Multi-Criteria Optimization:**

```
def calculate_route_score(route, weights):
    """
    Weighted scoring across multiple dimensions
    """

    # Time efficiency (0-1, lower is better)
    time_score = 1 - (route.duration / max_duration)

    # Cost efficiency (0-1, lower is better)
    cost_score = 1 - (route.fare / max_fare)

    # Comfort (0-1, higher is better)
    comfort_score = calculate_comfort(
        transfers=route.transfers,
        walking_distance=route.walking_distance,
        has_ac=route.has_air_conditioning
    )

    # Reliability (0-1, higher is better)
    reliability_score = 1 - (route.delay_risk / max_delay_risk)
```

```
# User preference match (0-1, higher is better)
preference_score = route.preference_score

# Weighted combination
total_score = (
    weights["time"] * time_score +
    weights["cost"] * cost_score +
    weights["comfort"] * comfort_score +
    weights["reliability"] * reliability_score +
    weights["preference"] * preference_score
)

return total_score

# Apply to all routes
for route in routes:
    route.recommendation_score = calculate_route_score(
        route,
        weights={
            "time": 0.25,
            "cost": 0.30, # Higher weight for cost (SLAIC 2025 focus)
            "comfort": 0.15,
            "reliability": 0.20,
            "preference": 0.10
        }
    )

# Sort by score
routes.sort(key=lambda r: r.recommendation_score, reverse=True)

# Mark top route as recommended
routes[0].is_recommended = True

return {
    "best_route": routes[0],
    "all_routes": routes,
    "total_routes_found": len(routes)
}
```

---

## Codebase Structure & Flow

### Backend Request Flow

#### Step-by-Step Execution

##### 1. USER SUBMITS ROUTE REQUEST

Mobile App → POST /api/v1/travel/plan-route

```
{
  "source": "Colombo Fort",
```

```
"destination": "Kandy",  
"mode": "transit",  
"user_id": "user_123"  
}
```

## 2. API GATEWAY (FastAPI)

File: backend/app/main.py

↓

- JWT Authentication (middleware/auth\_middleware.py)
- Rate Limiting Check (middleware/rate\_limiter.py)
- Request Validation (Pydantic models)

## 3. ROUTE HANDLER

File: backend/app/api/v1/travel\_routes.py:plan\_route()

↓

- Creates TravelState object
- Calls workflow.run\_travel\_agent()

## 4. LANGGRAPH WORKFLOW EXECUTION

File: backend/app/services/workflow.py

↓

Sequentially executes agents:

### 4a. INPUT PROCESSING AGENT

- Validates inputs
- Loads user preferences from MongoDB
- State: input\_processed

### 4b. MODE ROUTER AGENT

- Decision: "transit" → use transit\_aggregation
- State: routing\_transit

### 4c. TRANSIT AGGREGATION AGENT

- Google Maps API call for transit directions
- Finds 3 alternative routes
- State: routes\_aggregated

### 4d. FARE CALCULATION AGENT

- Queries MongoDB fare database

- Calculates total for each route
- State: fares\_calculated

#### 4e. FARE OPTIMIZATION AGENT

- Compares costs
- Identifies cheapest route
- Calculates savings
- State: fares\_optimized

#### 4f. PREFERENCE ANALYSIS AGENT

- Analyzes user history
- Scores routes based on past behavior
- State: preferences\_applied

#### 4g. LOCAL KNOWLEDGE AGENT

- Gemini LLM generates destination insights
- Adds cultural context
- State: context\_added

#### 4h. ROUTE OPTIMIZATION AGENT

- Multi-criteria scoring
- Ranks all routes
- Selects best route
- State: routes\_optimized

#### 4i. DISRUPTION MONITORING AGENT

- Checks weather API
- Queries community reports
- AI analyzes impact
- State: disruptions\_checked

#### 4j. RESPONSE COMPILATION

- Formats final response
- State: completed

### 5. RESPONSE TO CLIENT

File: backend/app/api/v1/travel\_routes.py

↓

Returns JSON:

```
{
  "status": "success",
  "request_id": "req_abc123",
  "processing_time": 4.2,
  "agents_used": [10 agent names],
  "response": {
    "best_route": {
      "route_id": "route_001",
      "duration_text": "3 hours 15 mins",
      "distance_text": "115 km",
      "estimated_cost": 180,
      "cost_currency": "LKR",
      "steps": [...],
      "recommendation_score": 0.87,
      "is_recommended": true
    },
    "all_routes": [...],
    "ai_disruption_analysis": "...",
    "destination_summary": "..."
  }
}
```

## 6. MOBILE APP RECEIVES RESPONSE

File: mobile-app/app/(main)/(tabs)/home.tsx:handlePlanRoute()

↓

- Transforms backend data to UI format
- Stores in AppContext
- Saves to route history (AsyncStorage)
- Navigates to routes screen

## 7. USER VIEWS ROUTE

File: mobile-app/app/(main)/(tabs)/routes.tsx

↓

- Displays best route with all details
- Shows AI recommendations
- Allows viewing alternatives
- Can save as favorite

# Database Operations

## MongoDB Collections Used

```
// User preferences lookup
db.user_preferences.findOne({ user_id: "user_123" })

// Route history storage
db.route_history.insertOne({
  user_id: "user_123",
  request_id: "req_abc123",
  source: "Colombo Fort",
  destination: "Kandy",
  routes_found: 3,
  selected_route: "route_001",
  timestamp: new Date()
})

// Fare database query
db.fares.find({
  type: "train",
  from: "Colombo Fort",
  to: "Kandy"
})

// Community reports
db.community_reports.find({
  location: { $near: route.coordinates },
  status: "active",
  created_at: { $gte: Date.now() - 2hours }
})

// Analytics tracking
db.analytics.insertOne({
  event: "route_planned",
  user_id: "user_123",
  agents_used: 10,
  processing_time: 4.2,
  mode: "transit"
})
```

## Redis Caching

```
# Cache user preferences (30 min TTL)
redis.setex(
  f"user_prefs:{user_id}",
  1800,
  json.dumps(preferences)
)

# Cache route results (5 min TTL)
redis.setex(
  f"route:{source}:{destination}:{mode}",
```

```
300,  
json.dumps(route_data)  
)  
  
# Rate limiting  
redis.incr(f"rate_limit:{user_id}:{current_minute}")  
redis.expire(f"rate_limit:{user_id}:{current_minute}", 60)
```

## Database Architecture

### MongoDB Schema Design

#### users Collection

```
{  
  _id: ObjectId("..."),  
  user_id: "user_123",  
  email: "john@example.com",  
  password_hash: "bcrypt_hash_here",  
  full_name: "John Doe",  
  phone: "+94771234567",  
  role: "user", // "user" | "admin" | "super_admin"  
  language_preference: "en", // "en" | "si" | "ta"  
  created_at: ISODate("2025-01-15T10:30:00Z"),  
  last_login: ISODate("2025-01-18T09:15:00Z"),  
  is_active: true,  
  profile_picture: "https:// ...",  
  verification_status: "verified"  
}
```

#### user\_preferences Collection

```
{  
  _id: ObjectId("..."),  
  user_id: "user_123",  
  preferred_transport_modes: ["train", "bus"],  
  max_budget_per_trip: 500, // LKR  
  max_walking_distance: 1.5, // km  
  accessibility_requirements: [],  
  comfort_preferences: {  
    prefer_ac: true,  
    prefer_express: true,  
    max_transfers: 2  
  },  
}
```

```
notification_settings: {
  disruption_alerts: true,
  price_drop_alerts: true,
  reminder_before_trip: 30 // minutes
},
privacy_settings: {
  share_location: true,
  anonymous_analytics: false
}
}
```

## route\_history Collection

```
{
  _id: ObjectId("..."),
  request_id: "req_abc123",
  user_id: "user_123",
  source: {
    address: "Colombo Fort Railway Station",
    lat: 6.9344,
    lng: 79.8428
  },
  destination: {
    address: "Kandy City Center",
    lat: 7.2906,
    lng: 80.6337
  },
  mode: "transit",
  routes_found: 3,
  selected_route_id: "route_001",
  selected_route_data: {
    duration_minutes: 195,
    distance_km: 115,
    estimated_fare: 180,
    transit_modes: ["train"],
    steps: [...]
  },
  agents_used: [
    "input_processing",
    "mode_router",
    "transit_route_aggregation",
    "fare_calculation",
    "fare_optimization",
    "user_preference_analysis",
    "local_knowledge_agent",
    "route_optimization",
    "disruption_monitoring",
    "response_compilation"
  ],
  processing_time_seconds: 4.2,
  timestamp: ISODate("2025-01-18T10:00:00Z"),
  is_favorite: false,
  trip_completed: false,
  user_rating: null
}
```

```
}
```

## community\_reports Collection

```
{
  _id: ObjectId("..."),
  report_id: "report_xyz789",
  user_id: "user_456",
  report_type: "traffic", // "traffic" | "delay" | "safety" | "fare" | "other"
  location: {
    address: "Baseline Road, Colombo",
    lat: 6.9271,
    lng: 79.8612,
    geohash: "tdnu20"
  },
  description: "Heavy traffic jam due to road construction",
  severity: "high", // "low" | "medium" | "high"
  affected_routes: ["route_138", "route_176"],
  media_attachments: ["https:// ...photo1.jpg"],
  upvotes: 15,
  downvotes: 2,
  verified: true,
  verified_by: "user_admin",
  verified_at: ISODate("2025-01-18T10:05:00Z"),
  status: "active", // "pending" | "active" | "resolved" | "dismissed"
  created_at: ISODate("2025-01-18T09:45:00Z"),
  expires_at: ISODate("2025-01-18T17:45:00Z")
}
```

## fares Collection (Sri Lankan Transit Pricing)

```
{
  _id: ObjectId("..."),
  fare_id: "fare_train_colombo_kandy",
  transport_type: "train",
  route_name: "Main Line",
  from_station: "Colombo Fort",
  to_station: "Kandy",
  distance_km: 115,
  fare_tiers: {
    third_class: 80,
    second_class: 150,
    first_class: 250,
    observation_saloon: 400
  },
  currency: "LKR",
  valid_from: ISODate("2025-01-01T00:00:00Z"),
  last_updated: ISODate("2025-01-15T00:00:00Z"),
}
```

```
source: "Sri Lanka Railways Official",
verified: true
}
```

## API Communication Flow

### Mobile App → Backend API

#### Authentication Flow

```
// mobile-app/src/services/api/auth.ts

// 1. User Registration
POST /api/v1/auth/register
Request: {
  email: "user@example.com",
  password: "SecurePass123",
  full_name: "John Doe",
  phone: "+94771234567"
}
Response: {
  user: {
    user_id: "user_123",
    email: "user@example.com",
    full_name: "John Doe"
  },
  access_token: "eyJhbGc...",
  refresh_token: "eyJhbGc...",
  token_type: "bearer"
}

// 2. Store tokens in AsyncStorage
await AsyncStorage.setItem('accessToken', response.access_token);
await AsyncStorage.setItem('refreshToken', response.refresh_token);

// 3. All future requests include token
GET /api/v1/user/profile
Headers: {
  Authorization: "Bearer eyJhbGc..."
}
```

#### Route Planning Flow

```
// mobile-app/src/services/api/travelService.ts
```

```
async planRoute(request: RouteRequest) {
  const response = await api.post('/travel/plan-route', {
    source: "Colombo Fort",
    destination: "Kandy",
    mode: "transit",
    user_id: this.userId,
    departure_time: "2025-01-18T10:00:00Z",
    preferences: {
      max_budget: 500,
      prefer_ac: true
    }
  });

  // Backend processes with 10 agents...

  return response.data;
}
```

## WebSocket Real-Time Updates

```
// mobile-app/src/services/websocket/client.ts

// Connect to WebSocket
const ws = new WebSocket('ws://localhost:8000/api/v1/ws');

ws.onmessage = (event) => {
  const update = JSON.parse(event.data);

  if (update.type === 'disruption_alert') {
    // Show notification
    showNotification({
      title: 'Route Disruption',
      message: update.message,
      severity: update.severity
    });
  }

  if (update.type === 'route_update') {
    // Update current route
    updateActiveRoute(update.route_data);
  }
};

// Send heartbeat every 30 seconds
setInterval(() => {
  ws.send(JSON.stringify({ type: 'ping' }));
}, 30000);
```

## Admin Dashboard → Backend API

```
// admin-dashboard/src/lib/api.ts

// Dashboard Overview
const dashboardAPI = {
  async getOverview() {
    const response = await axios.get('/admin/dashboard/overview', {
      headers: {
        Authorization: `Bearer ${adminToken}`
      }
    });

    return {
      total_users: response.data.total_users,
      active_users: response.data.active_users,
      total_trips: response.data.route_history_count,
      community_reports: response.data.total_community_reports,
      system_health: response.data.system_health,
      recent_activity: response.data.recent_activity
    };
  },

  // Send push notification to all users
  async broadcastNotification(message: string) {
    return await axios.post('/admin/notifications/broadcast', {
      title: 'System Announcement',
      body: message,
      target: 'all'
    });
  },

  // View AI agent performance
  async getAgentMetrics() {
    return await axios.get('/admin/agents/metrics');
  }
};
```



# Mobile App User Journey

## Complete User Flow

### 1. Onboarding (First-Time Users)

App Launch



Welcome Screen (app/(onboarding)/welcome.tsx)

- "Smart Transit Companion for Sri Lanka"
- Beautiful intro animation

↓

Features Screen (app/(onboarding)/features.tsx)

- "10 AI Agents for Best Routes"
- "Real-Time Disruption Alerts"
- "Multilingual Support"

↓

Permissions Screen (app/(onboarding)/permissions.tsx)

- Location: "To find routes near you"
- Notifications: "For disruption alerts"

↓

Language Selection

- English / සිංහල / தமிழ்

↓

Sign Up (app/(auth)/register.tsx)

- Email, Password, Name, Phone
- Creates account via API

↓

Preference Setup

- Preferred transport modes
- Budget constraints
- Accessibility needs

↓

Main App

## 2. Route Planning Journey

Home Tab (app/(main)/(tabs)/home.tsx)

↓

User enters:

- Source: "Colombo Fort" (with location autocomplete)
- Destination: "Kandy" (suggestions: Kandy City, Temple, etc.)
- Mode: Taps mode selector → Modal opens

↓

Mode Selection Modal (SriLankanModeSelector)

- 🚌 Bus (Local buses, affordable)
- 🚂 Train (Scenic, reliable)
- 🚚 Tuk Tuk (Quick, door-to-door)
- 🚗 Uber/PickMe (Comfortable, AC)
- 🚙 Own Vehicle (Driving directions)

- 🏍️ Motorbike (Fast, solo travel)
- 🚊 Mixed Transit (Best combination)
- 🚶 Walking (Short distances)

User selects: "🚊 Mixed Transit"

↓

Taps "🤖 Plan My Route"

↓

Multi-Agent Animation (MultiAgentAnimation.tsx)

- Visual progress showing each agent working:
  - ✅ Input Processing Agent - "Validating request..."
  - ✅ Mode Router Agent - "Determining route type..."
  - 🔄 Transit Aggregation Agent - "Finding bus & train combos..."
  - 🔄 Fare Calculation Agent - "Calculating costs..."
  - 🔄 Fare Optimization Agent - "Finding cheapest option..."
  - 🔄 Preference Analysis Agent - "Personalizing..."
  - 🔄 Local Knowledge Agent - "Adding Sri Lankan insights..."
  - 🔄 Route Optimization Agent - "Ranking routes..."
  - 🔄 Disruption Monitor Agent - "Checking for delays..."
  - ✅ All agents complete in 4.2 seconds!

↓

Auto-navigate to Routes Tab

↓

Routes Tab (app/(main)/(tabs)/routes.tsx)

- Shows "Best Route" card (AI recommended)
- Key info displayed:
  - 🚆 Train #1005 → Kandy
  - 🕒 3h 15min
  - 📏 115 km
  - 💰 LKR 180 (Second Class)
  - ★ 87% AI Confidence
  - 🎯 "Fastest & Most Reliable"
- Expandable details:
  - 📍 Step-by-step directions
  - 🚶 Walk to Colombo Fort Station (5 min)
  - 🚆 Board Train #1005 (Departure: 10:30 AM)
  - 🪑 Seats usually available
  - 🌄 Journey: 3h 10min
  - 📍 Arrive Kandy Station (1:40 PM)

 Tuk-tuk to destination (5 min, LKR 200)

- AI Insights:

 "This is the most scenic route - enjoy mountain views!"

 "Weather: Light rain expected, carry umbrella"

 "Temple of Tooth nearby, remove shoes at entrance"

- Alternative routes (tap to expand):

Route 2: Bus Route 1-1 (LKR 150, 4h 30min)

Route 3: Uber (LKR 12,500, 2h 45min)

↓

User can:

-  Save as favorite
-  Set departure reminder
-  View on map
-  Share with friends
-  Start navigation

### 3. During Journey

User taps "Start Navigation"

↓

Live Tracking Active:

- Current location updates every 30 seconds
- Progress bar shows journey completion
- Next step highlighted
- ETA continuously updated

↓

Real-Time Disruption Alert (WebSocket):

 Notification: "Train #1005 delayed 15 minutes"

↓

App shows:

 Updated ETA: 1:55 PM (was 1:40 PM)

 "Still fastest option, no action needed"

Alternative: "Bus Route 1-1 now available (LKR 150)"

↓

User completes journey

↓

Feedback Prompt:

"How was your trip?"

★★★★★ Rating

Optional comment

📷 Upload photo (for community)

#### 4. AI Chat Assistant

Chat Tab (app/(main)/(tabs)/chat.tsx)

↓

RAG-Powered Chatbot Interface

↓

User types: "How much is train ticket from Galle to Colombo?"

↓

Chatbot processes (RAG system):

1. ChromaDB vector search finds relevant docs
2. Google Gemini 2.0 Flash generates response
3. Includes real-time fare data from MongoDB

↓

Response:

🚂 Train fares from Galle to Colombo:

- Third Class: LKR 100
- Second Class: LKR 180
- First Class: LKR 300

Journey time: ~2.5 hours

💡 Tip: Morning trains (6-8 AM) are less crowded.

Book online via Sri Lanka Railways app to guarantee seats."

↓

User asks follow-up: "අද කාලගුණය කොහොමද?" (Sinhala: How's the weather today?)

↓

Chatbot (multilingual):

"අද කාලගුණය:

☁️ සිහින් වැහි අපේක්ෂා කරනවා

🌡️ උෂ්ණත්වය: 28°C

💧 සුළං: මධ්‍යස්ථ

☔ කුඩයක් රැගෙන යන්න!"

## 5. Community Contribution

Community Tab (app/(main)/(tabs)/community.tsx)

↓

User sees nearby reports:

📍 Heavy traffic on Galle Road (15 min ago)  
👍 12 upvotes | Verified ✓

📍 Bus Route 138 delayed (32 min ago)  
👍 8 upvotes | Verified ✓

↓

User wants to report:

Tap "➕ Report Issue"

↓

Report Form:

- Type: [Traffic / Delay / Safety / Fare / Other]
- Location: Auto-detected or manual entry
- Description: Text + voice input
- Photo: Optional upload
- Severity: [Low / Medium / High]

↓

Submits → Backend validates → Shown to nearby users

↓

Other users can:

- 👍 Upvote (increases credibility)
- 👎 Downvote (if incorrect)
- ✓ Admin verifies → becomes "Verified"



## Admin Dashboard Features

### Dashboard Overview Page

Login (admin@example.com)

↓

Dashboard Home (/dashboard)

↓

Stats Cards Display:

|                                                                                               |                                                                                               |                                                                                           |                                                                                             |
|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
|  Total Users |  Total Trips |  Reports |  System  |
| 1,247                                                                                         | 5,832                                                                                         | 234                                                                                       |  Healthy |
| 892 active                                                                                    | +15% this week                                                                                | 89 unverified                                                                             | All operational                                                                             |

Recent Activity Feed:

- user@example.com planned route (Colombo → Kandy) - 2 min ago
- admin verified community report #234 - 5 min ago
- New user registration: john@gmail.com - 8 min ago
- Push notification sent to 1,247 users - 15 min ago

System Health:

-  Database: Connected (17ms latency)
-  API Services: All operational
-  AI Agents: 10/10 active
-  External APIs: Google Maps ✓, Weather ✓

User Management

/dashboard/users



User Table (filterable, searchable):

| ID     | Email             | Role  | Last Login  | Status                                                                                         |
|--------|-------------------|-------|-------------|------------------------------------------------------------------------------------------------|
| usr_01 | john@example.com  | user  | 2 mins ago  |  Active   |
| usr_02 | admin@example.com | admin | 1 hour ago  |  Active   |
| usr_03 | inactive@ex.com   | user  | 30 days ago |  Inactive |

Actions:

- View user details & route history

- Edit user permissions
- Suspend/Activate account
- Reset password
- Delete user (with confirmation)

#### Advanced Filters:

- Role: All / User / Admin
- Status: All / Active / Inactive
- Registration date: Last 7 days / 30 days / All time
- Activity: Active users / Inactive users

## AI Agent Monitoring

/dashboard/agents



Real-Time Agent Activity:

Agent Performance Table:

| Agent Name            | Executions | Avg Time | Success |
|-----------------------|------------|----------|---------|
| Input Processing      | 5,832      | 0.12s    | 100%    |
| Mode Router           | 5,832      | 0.08s    | 100%    |
| Standard Route        | 2,145      | 1.2s     | 98.5%   |
| Transit Aggregation   | 3,687      | 2.1s     | 97.2%   |
| Fare Calculation      | 5,832      | 0.45s    | 99.8%   |
| Fare Optimization ★   | 5,832      | 0.67s    | 99.1%   |
| Preference Analysis   | 5,832      | 0.34s    | 100%    |
| Local Knowledge       | 5,832      | 1.5s     | 96.8%   |
| Route Optimization    | 5,832      | 0.89s    | 99.5%   |
| Disruption Monitoring | 5,832      | 0.56s    | 98.9%   |

Agent Workflow Visualization:

[Interactive Mermaid diagram showing agent flow]

- Click on any agent to see details
- View recent errors
- See input/output examples

## Recent Errors:

- Transit Aggregation: Google Maps API timeout (retry succeeded)
- Local Knowledge: Gemini rate limit hit (cached response used)

## Analytics Dashboard

/dashboard/analytics

↓

### Charts & Metrics:

#### 1. User Growth Chart (Recharts)

[Line chart showing daily new registrations]

#### 2. Route Planning by Mode

[Pie chart]

- Transit: 45%
- Train: 25%
- Bus: 15%
- Uber: 10%
- Other: 5%

#### 3. Popular Routes

[Bar chart - Top 10 routes]

1. Colombo → Kandy (1,234 searches)
2. Colombo → Galle (892 searches)
3. Kandy → Ella (567 searches)

#### 4. Cost Savings Impact

Total money saved by users: LKR 2,456,789

Average per trip: LKR 421

#### 5. Agent Processing Times

[Line chart showing avg processing time over last 30 days]

Trend: ↓ 15% faster (optimizations working!)

#### 6. User Engagement

- Daily Active Users: 892
- Weekly Active Users: 1,654
- Monthly Active Users: 2,341

- Avg routes per user: 4.7

#### Export Options:



Download CSV



Generate PDF Report



Email Report

## Push Notifications

/dashboard/notifications



Broadcast Interface:

Create New Notification:

Title: [System Maintenance Alert]

Message: [Servers will be down for... ]

Target:



All Users (1,247)



Active Users Only (892)



Specific User Segment

Schedule:



Send Now



Schedule for Later: [Date/Time]

[Preview] [Send Notification]

Notification History:

| Time       | Message            | Target | Delivered   |
|------------|--------------------|--------|-------------|
| 1 hour ago | Service disruption | All    | 1,247/1,247 |
| 2 days ago | New feature: Chat  | Active | 892/892     |
| 1 week ago | Maintenance done   | All    | 1,180/1,200 |

# Key Implementation Details

## 1. Environment Configuration

### Backend .env

```
# MongoDB Atlas (Cloud Database)
MONGODB_URL=mongodb+srv://TechBloom:TechBloom123@smarttransitcompanion...
DATABASE_NAME=transit_companion_db

# Google APIs (Core Features)
GOOGLE_MAPS_API_KEY=AIzaSyCM0C1y4kPjmrdcEOJAhqHi0wawe87Xfk8
GOOGLE_GEMINI_API_KEY=AIzaSyCzm456rBn00oMDUknEcIjh5Es6M5QPm7U

# AI Services
GROQ_API_KEY=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

# Observability
LANGFUSE_PUBLIC_KEY=pk-lf-53660d7c-7c18-4386-bc05-5959669d214d
LANGFUSE_SECRET_KEY=sk-lf-d1b3a474-a7ff-4273-9469-f537c801909b
LANGFUSE_HOST=https://cloud.langfuse.com

# Weather
OPENWEATHER_API_KEY=f41125c406e0ea4ab65167eaa495879e

# Search
SERPER_API_KEY=11bfa639bbffa54727f38682791619bcd8be56c0

# Security
SECRET_KEY=your-secret-key-here-change-in-production
BACKEND_CORS_ORIGINS=["*"]

# App Settings
DEBUG=true
LOG_LEVEL=INFO
MAX_ROUTES_PER_REQUEST=5
DEFAULT_SEARCH_RADIUS_KM=50
```

### Mobile App .env

```
# Backend Connection
EXPO_PUBLIC_API_HOST=10.47.78.83 # Local network IP
EXPO_PUBLIC_API_PORT=8000
EXPO_PUBLIC_API_PROTOCOL=http

# WebSocket
EXPO_PUBLIC_WS_PROTOCOL=ws
```

```
# For different environments:  
# - Android Emulator: 10.0.2.2  
# - iOS Simulator: localhost  
# - Physical Device: Your computer's local IP
```

## 2. Startup & Deployment

### Backend Startup

```
# Install dependencies  
cd backend  
pip install -r requirements.txt  
  
# Run application  
python run.py  
  
# Server starts on http://0.0.0.0:8000  
# API docs available at http://localhost:8000/docs
```

### Admin Dashboard Startup

```
cd admin-dashboard  
npm install  
npm run dev  
  
# Opens on http://localhost:3000
```

### Mobile App Startup

```
cd mobile-app  
npm install  
npx expo start  
  
# Options:  
# - Press 'a' for Android emulator  
# - Press 'i' for iOS simulator  
# - Scan QR code with Expo Go app  
# - Press 'w' for web browser
```

### 3. Authentication Flow

```
# backend/app/core/security.py

from passlib.context import CryptContext
from jose import jwt

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=30)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm="HS256")
    return encoded_jwt

def verify_password(plain_password, hashed_password):
    return pwd_context.verify(plain_password, hashed_password)

def get_password_hash(password):
    return pwd_context.hash(password)
```

### 4. RAG Chatbot Implementation

```
# backend/app/chatbot/chatbot.py

from langchain_chroma import Chroma
from langchain_google_genai import GoogleGenerativeAIEmbeddings, ChatGoogleGenerativeAI

# Initialize embeddings
embeddings = GoogleGenerativeAIEmbeddings(
    model="models/embedding-001",
    google_api_key=GOOGLE_GEMINI_API_KEY
)

# Load vector database
vectordb = Chroma(
    persist_directory="./db",
    embedding_function=embeddings
)

# Initialize LLM
llm = ChatGoogleGenerativeAI(
    model="gemini-2.0-flash-exp",
    temperature=0.2
)

# Create RAG chain
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
```

```
retriever=vectordb.as_retriever(search_kwargs={"k": 3}),
return_source_documents=True
)

# Query
def ask_question(question: str):
    result = qa_chain({"query": question})
    return {
        "answer": result["result"],
        "sources": result["source_documents"]
    }
```

## SLAIC 2025 Competition Alignment

### Competition Requirements vs Implementation

| Requirement            | Status             | Implementation                                                                                                                                                                                             |
|------------------------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Minimum 7 AI Agents    | ✓ 10 Agents        | Input Processing, Mode Router, Standard Route, Transit Aggregation, Fare Calculation, <b>Fare Optimization (special)</b> , Preference Analysis, Local Knowledge, Disruption Monitoring, Route Optimization |
| Sri Lankan Focus       | ✓ Deep Integration | Railway API, NTC Buses, Tuk-tuk pricing, Cultural context, Sinhala/Tamil support                                                                                                                           |
| Community Data         | ✓ Full Platform    | Crowdsourced reports, verification system, upvoting, admin moderation                                                                                                                                      |
| Real-time Intelligence | ✓ Multi-source     | Live traffic, weather API, WebSocket updates, predictive analytics                                                                                                                                         |
| Production Ready       | ✓ Enterprise Grade | FastAPI + MongoDB + Redis, JWT auth, rate limiting, error handling                                                                                                                                         |
| Innovation             | ✓ Multi-Agent AI   | LangGraph orchestration, RAG chatbot, fare optimization algorithms                                                                                                                                         |

### Unique SLAIC 2025 Features

#### 1. Fare Optimization Agent (Agent #6)

Competition Highlight: AI-Powered Cost Minimization

How it works:

- Analyzes all route alternatives
- Compares total costs including transfers
- Identifies hidden savings (e.g., cheaper bus instead of taxi)
- Calculates time vs cost trade-offs
- Learns from user behavior to balance savings with convenience

Example Impact:

Regular route: Uber Colombo → Kandy = LKR 12,500

Optimized: Train (LKR 180) + Tuk-tuk (LKR 200) = LKR 380

Savings: LKR 12,120 (96% cheaper!)

Time difference: +30 minutes

AI Recommendation: "Save LKR 12,120 with just 30 min extra time"

## 2. Sri Lankan Cultural Integration

Local Knowledge Agent provides:

- Temple etiquette (remove shoes, dress modestly)
- Festival impact on routes (Vesak, Poson, etc.)
- Peak hour warnings (office hours, school times)
- Language support for announcements
- Local landmark navigation

Example:

Destination: Anuradhapura

AI adds: "Sacred city - dress conservatively.

Best visit early morning (6-8 AM) to avoid heat.

Poya Day = very crowded, book train tickets in advance."

## 3. Community-Driven Intelligence

Real-time crowdsourced data:

- Traffic jam reports (verified by multiple users)
- Bus/train delay confirmations
- Safety alerts for travelers
- Fare discrepancy reporting
- Accessibility information

Impact:

User reports "Heavy traffic on Galle Road"

- 15 other users confirm
- Admin verifies
- Disruption Monitor Agent factors into routing
- Alternative routes suggested
- All users save time

## **Competitive Advantages**

### **1. Most AI Agents (10 vs required 7)**

- Deepest intelligence in route planning
- Comprehensive multi-criteria optimization

### **2. Production-Ready Full Stack**

- Mobile app (React Native)
- Admin dashboard (Next.js)
- Backend API (FastAPI)
- All components fully integrated

### **3. Advanced RAG Chatbot**

- Multilingual (3 languages)
- 50+ document knowledge base
- Real-time data integration

### **4. Enterprise Architecture**

- MongoDB Atlas (cloud database)
- Redis caching
- JWT authentication
- Rate limiting
- Error handling
- Logging & monitoring

### **5. Real Sri Lankan Integration**

- Not just theoretical - actual rail/bus data
- Local language support (not just English)
- Cultural awareness (festivals, customs)
- Community validation



## Project Statistics

### Codebase Metrics

- **Total Lines of Code:** ~12,000+ lines
- **Backend Python:** ~8,000 lines
- **Frontend TypeScript:** ~4,000 lines
- **AI Agent Code:** 101 KB (agent\_nodes.py)
- **API Endpoints:** 11 route files
- **Database Collections:** 17 collections
- **External APIs:** 7 integrations

### Features Count

- **AI Agents:** 10 specialized agents
- **Transport Modes:** 8 modes (bus, train, tuk-tuk, uber, driving, bike, walking, mixed)
- **Languages:** 3 (English, Sinhala, Tamil)
- **Database Collections:** 17
- **API Endpoints:** 50+
- **Mobile Screens:** 15+
- **Admin Pages:** 7

### Technology Stack

- **Backend:** FastAPI, Python 3.11+, LangGraph, LangChain
- **AI/LLM:** Google Gemini 2.0 Flash, Groq, Langfuse
- **Databases:** MongoDB Atlas, ChromaDB, Redis
- **Frontend:** React 19, Next.js 15, React Native 0.81
- **Mobile:** Expo SDK 54, TypeScript, NativeWind
- **APIs:** Google Maps, OpenWeather, Serper, Sri Lanka Railways



## Project Impact & Vision

### For Commuters

- **Time Savings:** Faster route discovery

- **Cost Savings:** 20-40% cheaper routes through optimization
- **Reduced Stress:** Predictable journeys with real-time updates
- **Language Accessibility:** No barriers for local language speakers

## For Transportation Ecosystem

- **Data-Driven Insights:** Analytics on passenger patterns
- **Service Improvement:** Feedback loop for operators
- **Community Safety:** Crowdsourced safety information
- **Innovation Platform:** Foundation for future smart city features

## For Sri Lanka

- **Technology Leadership:** World-class AI application
- **Economic Impact:** Reduced commuting costs
- **Environmental:** Optimized routes = reduced emissions
- **Social Inclusion:** Accessibility features for all



## Future Enhancements

### Phase 2 Roadmap

1. **Voice Interface:** Full voice-controlled navigation
2. **AR Navigation:** Augmented reality wayfinding
3. **Offline Mode:** Download routes for offline use
4. **Social Features:** Share routes, travel together
5. **Gamification:** Rewards for eco-friendly choices
6. **B2B Integration:** API for transport companies
7. **Government Partnership:** Official transit authority integration



**Document Version:** 1.0 🏆 **Competition:** SLAIC 2025 - Transportation & Logistics  
(Use Case 02) 👥 **Team:** TechBloom ✉️ **Contact:** [Competition Contact Information]

---

**Built with ❤️ in Sri Lanka for Sri Lankans Powered by 10 AI Agents, Orchestrated  
by LangGraph Production-Ready Full-Stack Platform**